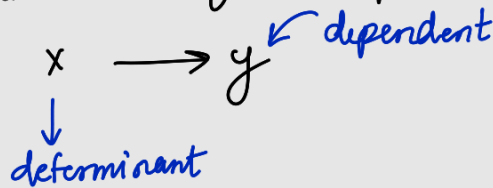


DBMS Module 3

Functional Dependencies

One variable / attribute is dependent on another. Using the variable x , we can find the value of the dependent variable y from the table.



eg: STUDENT FD ✓

R-no	Name
1	Ankit
2	Anjana
3	Raj
4	Sachin

STUDENT FD ✗

R-no	Name
1	Ankit
2	Anjana
3	Raj
4	Sachin
3	Anand

not FD

Conditions for Functional Dependency:

If $t_1 x = t_2 x$ then $t_1 y = t_2 y$

Q: Check whether the given are functionally dependent

EmpID	Name	Joining Year	Dept	Salary
001	Aakash	2010	IT	50000
002	Nandini	2026	HR	25000
003	Aakash	2023	IT	20000
004	Jash	2020	Finance	50000

1. EmpID $\rightarrow$ Name Yes, it is functionally dependent.

2. Name $\rightarrow$ EmpID No

3. EmpID $\rightarrow$ Joining Year Yes

4. Joining Year $\rightarrow$ EmpID Yes

5. Dept $\rightarrow$ Salary No

6. Salary $\rightarrow$ Dept No.

We can group set of attributes and check functional dependencies.

eg: (EmpID, Name) $\rightarrow$ Salary ✓ FD

(Name, Dept) $\rightarrow$ (Joining Year, Salary) ✗

• Normalization

Process to streamline database data correctly. To reduce data redundancy and dependency. We can decompose the tables into smaller tables.

eg: STUDENT

R-no	Name	Dept	Teacher	Code
1	Arathi	CS	Anna	CS101
2	Anju	ME	Meera	ME201
3	Aswin	CS	Anna	CS101

← Addition of HD details

← repetition of information

(i) Insertion Anomaly

Insertion is not possible when some required info is missing

eg: Adding only Dept, Teacher, Code

(ii) Updation Anomaly

Modification of data leads to inconsistency

eg: changing code for one dept must reflect everywhere.

(iii) Deletion Anomaly

Unintentional deletion of record leads to loss of data.

eg. deletion of 1 student removes ME record.

STUDENT

PK R-no	Name	Dept	Teacher	Code
1	Arathi	CS	Anna	CS101
2	Anju	ME	Meera	ME201
3	Aswin	CS	Anna	CS101

STUDENT

R-no PK	Name	Dept FK
1	Arathi	CS
2	Anju	ME
3	Aswin	CS

DEPARTMENT

Dept PK	Teacher	Code
CS	Anna	CS101
ME	Meera	ME201

• First Normal Form (1NF)

Conditions to check 1NF:

1. No column with multivalued attributes, composite attributes
2. Column should contain value from same domain
3. Column should have unique name.
4. No ordering to rows and columns
5. No duplicate rows.

PK

R-no	Name	Elective
1	Aarush	C/C++
2	Siva	Java
3	Rahul	DBMS

Solution 1
→

PK

R-no	Name	Elective
1	Aarush	C
1	Aarush	C++
2	Siva	Java
3	Rahul	DBMS

Solution 2
↓

R-no	Name	Elective1	Elective2
1	Aarush	C	C++
2	Siva	Java	Null
3	Rahul	DBMS	Null

Solution 3
↘

PK

R-no	Name
1	Aarush
2	Siva
3	Rahul

FK

R-no	Elective
1	C
1	C++
2	Java
3	DBMS

• Second Normal Form (2NF)

Conditions to check 2NF:

1. It is in 1NF

2. Has no partial dependency

R-no, Teacher, Subject

eg: 1) $R(A, B, C, D, E, F)$ FD: $\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow E\}$

CK: AF Not in 2NF

2) $R(A, B, C, D)$ FD: $\{AB \rightarrow C, C \rightarrow D\}$

ck: AB Not in 2NF

Proper subset of candidate key → non-prime attribute

3) $R(A, B, C, D)$

FD: $\{AB \rightarrow CD, C \rightarrow A, D \rightarrow B\}$

CK: AB, CB Is in 2NF

Steps to check PD:

1. Take closure of all the attributes. It is the super key RHS has all attributes
 $\hookrightarrow$ itself, what can be derived out of it.
2. Try to find CK by discarding the attributes that can be derived from.
3. Find the closure of the new set of attributes. Find closure of its proper set. Check whether its super key. If yes, repeat steps 2 and 3.
4. If we find the CK, check whether PA are present on the right side, substitute them and check for the new CKs.
5. Check for the PD rule with Prime attributes (PA)

• Third Normal Form (3NF)

A table is in 3NF if and only if for each of its non-trivial functional dependency at least 1 of the following conditions holds.

- (i) LHS is SK
- (ii) RHS is prime attribute

Conditions to check 3NF:

1. It is in 2NF.
2. No transitive dependency for non-prime attributes.

$A \rightarrow B, B \rightarrow C$ then $A \rightarrow C$
 $\underbrace{\hspace{10em}}$
NPA

eg: 1. $R(A, B, C, D)$

FD: $(A \rightarrow B, B \overset{x}{\rightarrow} C, C \overset{x}{\rightarrow} D)$

SK and CK: A Not in 3NF

2. $R(A, B, C, D, E, F)$

FD: $(AB \rightarrow CDEF, BD \rightarrow F)$

CK: AB Not in 3NF

Transitive Dependency

$NPA \rightarrow NPA$

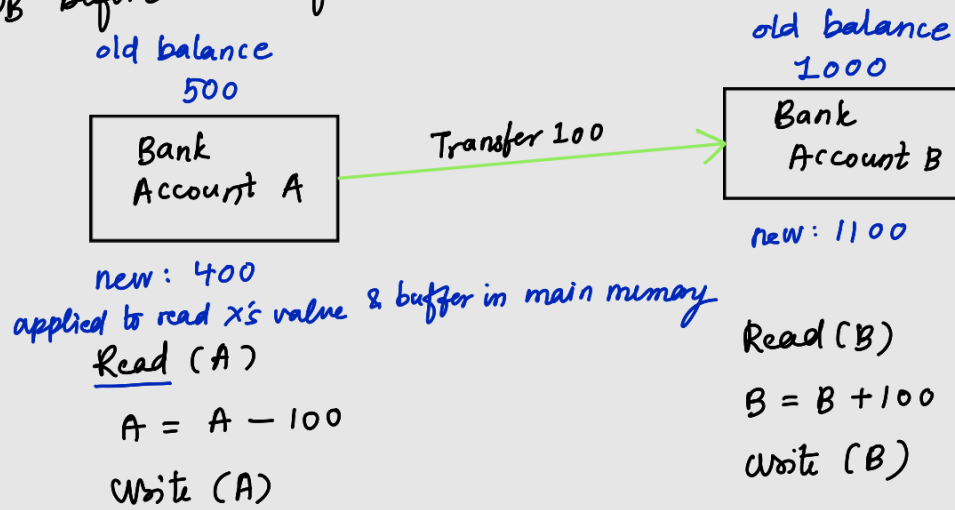
Partial Dependency

$PA \rightarrow NPA$

• Transaction Management

Program unit whose execution may change the content of database.
eg: Bank balance changing when you transfer money.

DB before and after transaction must be in **consistent** state.



Before transaction: $A + B = 500 + 1000 = 1500$

After transaction: $A + B = 400 + 1100 = 1500$

Transactions need to satisfy ACID properties

A : Atomicity
C : Consistency
I : Isolation
D : Durability

• Transaction Processing:

Transaction is made to change data in a database which can be done by inserting new data, updating / deleting data.

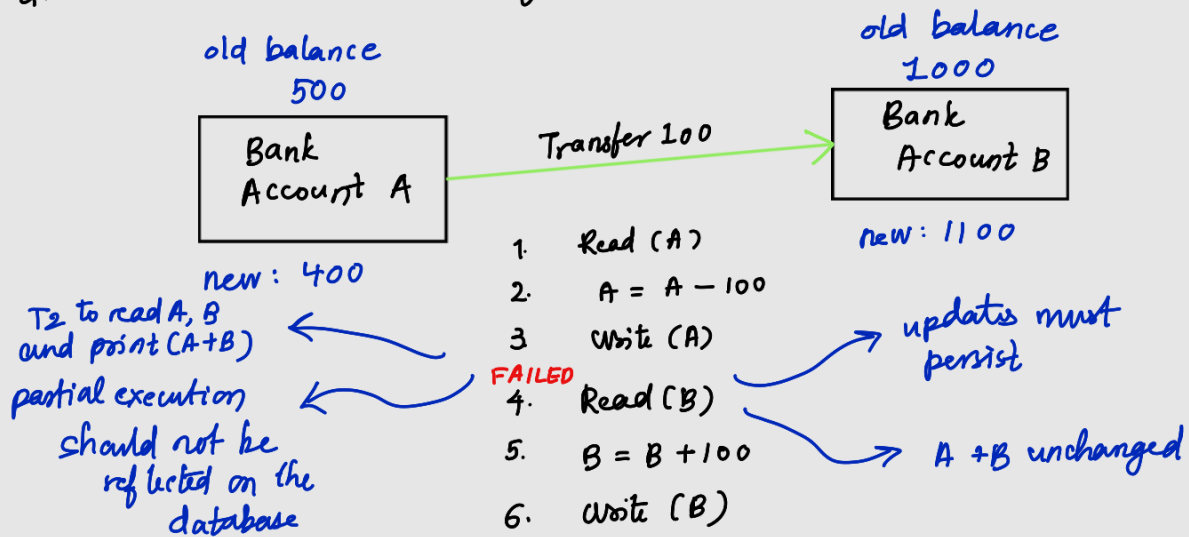
The operations performed are:

1. Read(X): to read the value of X from database server and buffer it to the main memory.
2. Write(X): to write the value of X to the database server from buffer.
3. Insert
4. Delete
5. Update

characteristics: consists of multiple operations, executed as a single unit, can be fully completed or fully aborted.

Main issues:

1. Failures such as hardware failures, system crashes, logical errors
2. Concurrent execution of multiple transactions



• Properties of transaction:

ACID properties

Atomicity: Either all operations of transaction are properly reflected in the database or none are.

Consistency: Execution of transaction in isolation preserves consistency and it maintains valid state.

Isolation: Each transaction must be unaware of other concurrently executed transactions.

eg: $T_1, T_2 \longrightarrow T_1$ finished execution, then T_2 starts.

Durability: Change made to the database must persist even if there are system failures.

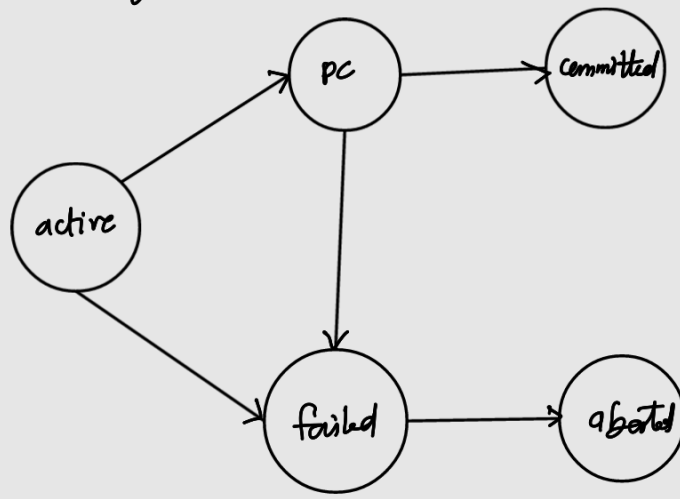
• Transaction state

1. Active: Initial state, transaction stays in this state while it is executing
2. Partially committed: after the final statement has been executed.
3. Failed: After the discovery that normal execution can no longer proceed.
4. Aborted: After transaction has been rolled back and the database restored to its state prior to start of transaction

RESTART

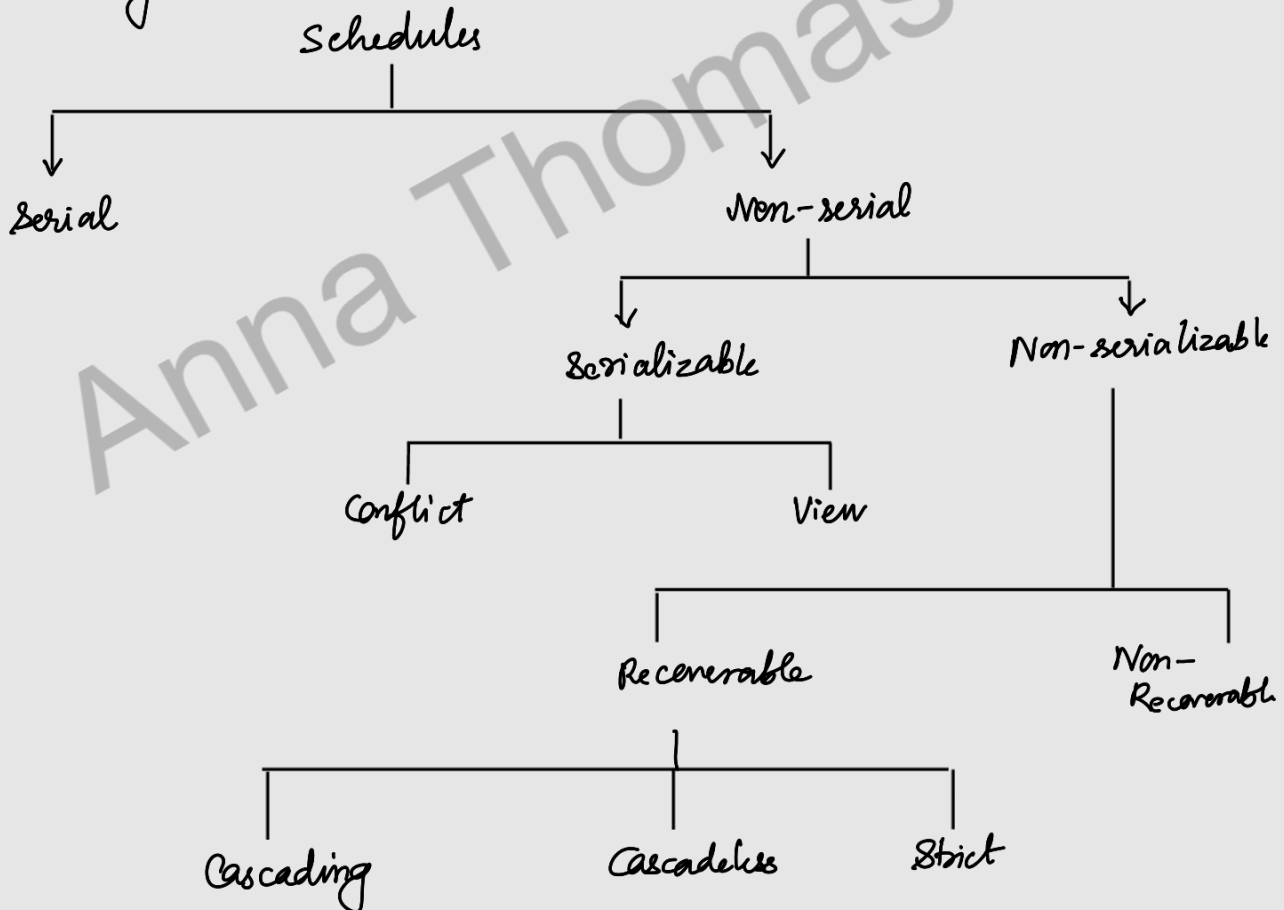
KILL

5. Committed: Successful completion.



• Schedule

Series of operation from one transaction to another transaction.
Used to preserve the order of the operation in each individual transaction.
It is the arrangement of transaction operations.



1) Serial Schedules

Executed serially. No transaction is executed in parallel.

Always consistent, recoverable, cascadeless, strict. It is non-interleaved.

It always gives the correct result.

eg:

T ₁	T ₂
R(A) W(A) R(B) W(B) Commit	 R(A) Commit

there exists $n!$ different valid serial schedules.

eg: 2 transactions can be done in $T_1 \rightarrow T_2$ or $T_2 \rightarrow T_1$

2) Non-serial schedules

Multiple transactions execute concurrently. They are interleaved or mixed with each other.

Not always consistent, recoverable, cascadeless, strict

eg:

T ₁	T ₂
R(A) W(A) Commit	 R(B) W(B) R(A) Commit

Total no. of non serial schedules = Total no. of schedules - total no. of serial schedules.

• Serializability

Helps to identify which non-serial schedules are correct and will maintain the consistency of the database.

A serializable schedule always leaves database in consistent state. It can have multiple transactions executed concurrently. It improves resource utilization and CPU throughput.

1) Conflict Serializability

Swapping its non conflicting operation converted non-serial to serial schedule.

If:

1. Both operations belong to different transactions.
2. Both operations are on same data item
3. Atleast one of it is a write operation

Then we can call them conflicting operations.

eg: 1-

T ₁	T ₂
Read (X)	Read (X)
Read (X)	Write (X)
Write (X)	Read (X)
Write (X)	Write (X)



2.

T ₁	T ₂
Read (X)	Read (Y)
Read (X)	Write (X)
	Read (X)

2) View Serializability

Check whether given schedule is view serializable / not.
If a given schedule is found to be view equivalent to some serial schedule, then it is called view serializable schedule.

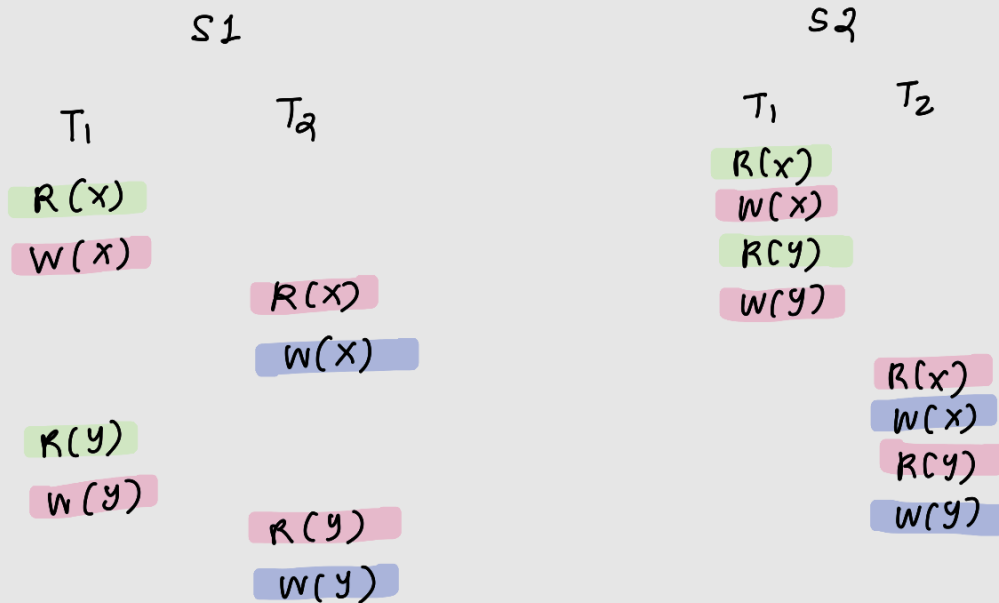
For T₁ and T₂ to be view equivalent:

1. Initial read of each data item must be match in both schedules.

2. Final write on each data item must be match in both schedules.

3. In S₁, the T₁ is reading a data item updated by T₂, then in S₂, T₁ should read the data item after write operation by T₂.

eg:



• Recoverability

Ensures that after a failure, the database can restore a consistent state by keeping committed change and undoing uncommitted ones.

$$(T_1 \rightarrow T_2)_{\text{reads}} \implies (T_1 \xrightarrow{\text{commits}} T_2)_{\text{commits}}$$

1) Cascading schedule

Failure of one transaction causes several other dependent transactions to roll back / abort.

Leads to wastage of CPU time

eg:

T1	T2	T3
R(A)		
W(A)		
	R(A)	
	W(A)	
		R(A)
		W(A)
FAILURE		

If committed, the schedule would have been irrecoverable.

2) Cascadefree schedule

Not allowed to read item until last transaction that has written it is committed or aborted.

Allows only committed read operations and hence avoids cascading rollbacks. It allows uncommitted write operations.

eg:

T_1	T_2
$R(A) \checkmark$	
	$W(A) \longrightarrow$ no need to commit
Commit	

• Concurrency Control

Process of managing simultaneous operations on the database without having them interfere with one another. Enhances overall CPU utilization. Produces correct result without violating data integrity of the database.

Issues with concurrency:

1. Loss update problem: multiple transaction select and update same row.

eg:

T_1	T_2
$R(A)$	
$A = A - 50$	
$W(A)$	
	$A = 100$
	$R(A)$
	$A = A + 100$
	$W(A)$

2. Dirty read problem: if a transaction fails, before rollback, the data item is accessed by another transaction.

eg:

T_1	T_2
$R(A)$	
$A = A + 50$	
$W(A)$	
Failure	
	$R(A)$

rollback

3. Inconsistent analysis problem: Two different values are read for the same database item.

eg:

T_1	T_2
$R(A)$	
	$R(A)$
	$A = A + 100$
	$W(A)$
$R(A)$	

- Two Phase Locking Technique: (2PL)

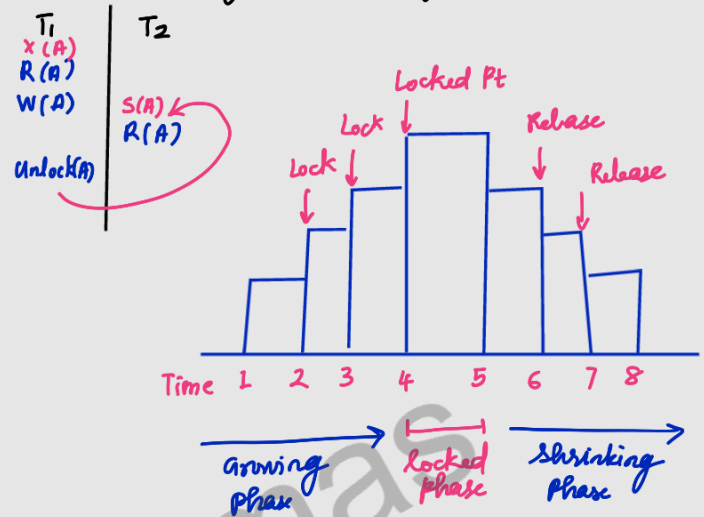
Growing Phase
Locks are acquired on the data items
no locks released

Shrinking Phase
Existing locks can be relaxed.
no new locks

Lock point is the point where a transaction has achieved its final lock. It is the point where growing phase ends and shrinking phase begins.

eg:

T1	T2
lock S(A)	lock S(A)
lock X(B)	
unlock (A)	
	lock X(C)
unlock (B)	
	unlock (A)
	unlock (C)



• Database Recovery Management:

Process of restoring database to the most consistent state that existed just before the failure.

1. Pre condition
consistent state

2. Condition
occurs a system
failure

3. Post condition
restore the database

- Types of failures:

1. System failure: OS fault, RAM failure
2. Transaction failure: Deadlock, i/p related issues
3. Media failure: Disk head crash, power failure

- strategies:

LOG BASED RECOVERY

1. Deferred update: (REDO operation / Rollback if not committed)

Transactional operations don't update the database immediately. It

is physically updated only after it reaches its commit point.

eg:

T1
R(A)

A = 100
B = 200

$A = A + 100$
 $W(A)$
 $R(B)$
 $B = B + 50$
 $W(B)$
 Commit

2. Immediate update:

Transactional operations update the database immediately during execution before reaching the commit pt. On abort/failure, a rollback needs to be done to restore the database to consistent state.

eg:
 T_1
 $R(A)$
 $A = A + 100$
 $W(A)$
 $R(B)$
 $B = B + 50$
 $W(B)$
 Commit

$A = 100$ $B = 200$

3. Shadow Paging

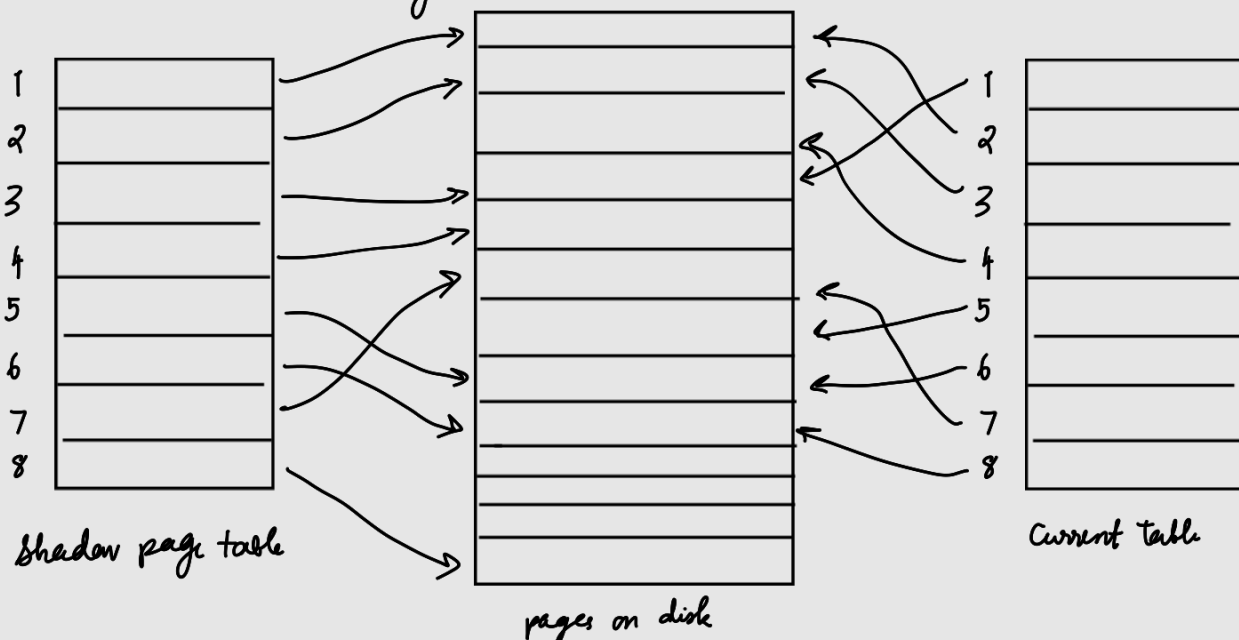
Alternative to log-based recovery.

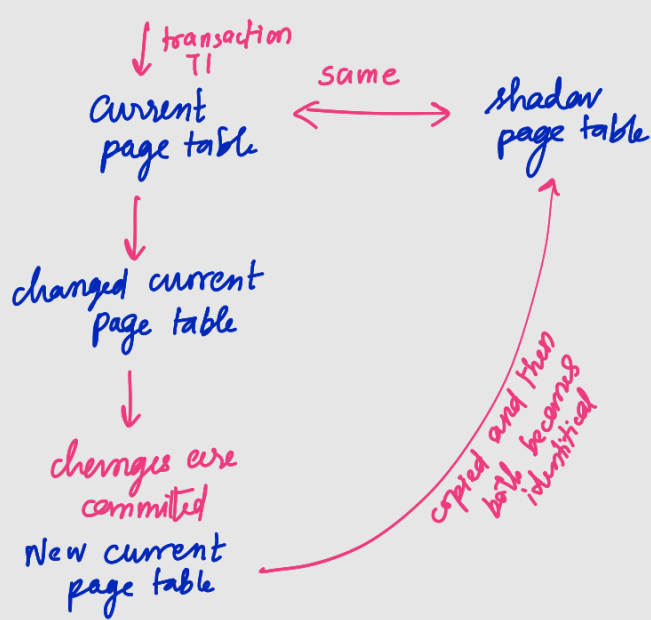
Database is partitioned to no. of fixed length blocks $\rightarrow$ pages which are not stored any particular order. For finding a given page, we use a page table.

- $\rightarrow$ current page table (changes)
- $\rightarrow$ shadow page table (never changes during transactions & after commit, both become same)

Each entry contains a pointer to page on disk.

When transaction begins, both the tables are identical.





When system comes back for further transactions, it copies the updated shadow page table in the memory and uses it as current page table.

Advantages:

1. No log record required.
2. Faster recovery.

Disadvantages:

1. Commit overhead (disk address)
2. Data fragmentation
3. Garbage collectors (old info replaced)

Anna Thomas